

IA 642, Defensive Security Week 04, Module 02, Lab 4.2

Standing Up Meridian's SOC: Wazuh, Kerberoasting Telemetry, and a Rule That Actually Fires

Detection Engineering, SIEM & SOC Operations Hands-On Build

Format: Individual, hands-on you deploy and operate a real SIEM

Total time: 3-4 hours (SIEM install alone takes 20-40 minutes of wait time)

Deliverable: A lab report (Word or PDF) and a short presentation with captured evidence, your rule XML, and written answers see Section 8

0. Before You Start

Module 01's Lab 4.1 kerberoasted meridian.local: a weak service account, a burst of TGS requests, an offline crack of an RC4-encrypted ticket. Nobody at Meridian caught it, because nothing was watching the domain controller's Security log for the pattern. This lab closes that gap for real. You will stand up a single-node Wazuh SIEM, feed it real telemetry from real hosts, write a detection rule from scratch, and watch it fire against the exact behavior Module 01 produced.

ENVIRONMENT CONTINUITY

If you completed Lab 4.1, you already have MERIDIAN-DC01 (the domain controller) and MERIDIAN-RED01 (the Kali attacker box) running on your isolated lab network reuse them as-is. You do not need to repeat Lab 4.1's Kerberoasting attack from scratch; Section 4 below tells you exactly what to re-run. If you have not completed Lab 4.1, Section 1.2 gives a brief fallback: the minimum steps to get one Windows host and one Linux host reachable on the same isolated network, without redoing all of Lab 4.1.

Learning Outcomes

By the end of this lab, you will be able to:

- Deploy a single-node Wazuh manager, indexer, and dashboard, and verify it is collecting cleanly

- Register Windows and Linux hosts as Wazuh agents and confirm active check-in
- Generate real, detectable telemetry a Kerberos TGS-request burst or an SSH brute-force run
- Write a real Wazuh detection rule in local_rules.xml, correctly mapped to a MITRE ATT&CK technique ID
- Deploy that rule and prove empirically, in the dashboard, that it fires on the behavior it targets
- Triage a batch of realistic alerts TP/FP, ATT&CK mapping, escalate/close/tune and compute MTTD
- Tune a rule to eliminate a specific stated false positive without losing detection of the original attack

Scope, Safety, and Academic Honesty

Everything in this lab happens inside virtual machines you build and own, on an isolated internal network with no route to the internet during the attack exercises in Sections 4 and 5. The Kerberoasting and SSH brute-force techniques in this lab target only hosts you built for this course. Do not point either technique at any host, account, or network you do not own or do not have explicit written authorization to test university systems and employer systems included.

Prerequisites

You should already be able to, from this week's Module 01 and Module 02 lectures:

- Explain what a Kerberos TGS request is and why Event ID 4769 is logged on the domain controller
- Recognize MITRE ATT&CK T1558.003 (Kerberoasting) and T1110 (Brute Force) by name and tactic
- Read a Sigma-style detection rule and explain what a Wazuh rule's if_sid / field / frequency elements do
- Distinguish Tier 1 triage from Tier 2 investigation, and state what MTTD measures

You will also need basic comfort with a Linux shell and Windows PowerShell, this lab will not re-teach either, but every command is given exactly as typed.

1. Lab Environment Setup

This lab adds one new host to your lab network: MERIDIAN-SOC01, an Ubuntu host running Wazuh. It reuses MERIDIAN-DC01 and MERIDIAN-RED01 from Lab 4.1 as telemetry sources. All three hosts sit on the same isolated VirtualBox/hypervisor Internal Network you built in Lab 4.1 (or the Week 03 bonus lab, if that is the network you already have configured).

1.1 If You Completed Lab 4.1

- Power on MERIDIAN-DC01 and MERIDIAN-RED01 and confirm both still have working static IPs on the internal network.
- Confirm MERIDIAN-DC01 can resolve and ping MERIDIAN-RED01 and vice versa if the internal network name or promiscuous-mode setting drifted, fix it now before continuing.
- Skip to Section 1.3 to build MERIDIAN-SOC01.

1.2 Fallback: Minimal Two-Host Setup (Skip If You Did 1.1)

You do not need to rebuild all of Lab 4.1's Active Directory domain to complete this lab; you need one Windows host that can log Kerberos-style Security events and one Linux host to use as the attacker/second agent. This is the minimum path.

- Build the isolated network. In VirtualBox: File > Tools > Network Manager > Internal Networks, create "meridian-lab-net" if it does not already exist (identical to the Week 03 bonus lab's network).
- Windows host (stand-in for MERIDIAN-DC01). A Windows Server 2022 or Windows 10/11 Enterprise evaluation VM, 4 GB RAM, attached to meridian-lab-net with a static IP. You do not need to promote it to a domain controller for this lab. Part C's fallback (SSH brute force) does not require Kerberos at all, and if you want to try the primary Kerberoasting path, promote it to a DC and create one SPN-bearing service account exactly as Lab 4.1 instructs.
- Linux host (stand-in for MERIDIAN-RED01). Kali Linux or Ubuntu, 2 GB RAM, attached to meridian-lab-net with a static IP, sshpass and impacket-scripts installed (apt install -y sshpass python3-impacket on Kali; pipx install impacket on plain Ubuntu).
- Verify both hosts can ping each other across meridian-lab-net before continuing.

1.3 Build MERIDIAN-SOC01 (Ubuntu)

1. Create a new VM: Ubuntu Server 22.04 or 24.04 LTS (64-bit), 8192 MB RAM minimum, 4 vCPU recommended, 50 GB dynamically allocated disk.
2. Attach two network adapters: Adapter 1 as NAT (for package downloads only you will not disable this one, since the dashboard is reached over the internal network but package installs need internet), Adapter 2 as Internal Network "meridian-lab-net", same as your other two hosts.
3. Install Ubuntu Server from the ISO. A minimal install with OpenSSH server enabled is sufficient.
4. After install, identify the internal-network interface and give it a static IP via Netplan.

```
ip -br addr show
# Note the interface name attached to meridian-lab-net (commonly enp0s8)
# replace it below if yours differs.

sudo tee /etc/netplan/01-meridian-soc01.yaml > /dev/null <<'EOF'
network:
  version: 2
  ethernets:
    enp0s8:
      addresses: [192.168.56.30/24]
EOF
sudo netplan apply
ip -br addr show enp0s8
```

Run on MERIDIAN-SOC01. Adjust the subnet to match your existing lab network's addressing.

Confirm MERIDIAN-SOC01 can ping both MERIDIAN-DC01 and MERIDIAN-RED01 (or your fallback hosts) across meridian-lab-net before continuing to Part A.

2. Part A Deploy the SIEM

Wazuh's official quickstart installs the manager, indexer, and dashboard together on a single host exactly the architecture Module 02's lecture diagrammed, now as a real deployment. This single-node profile is rated by Wazuh for up to roughly 100 agents and 90 days of indexed alert data, comfortably enough for this lab.

Step 1 Run the All-in-One Installer

```
# On MERIDIAN-SOC01:
curl -sO https://packages.wazuh.com/4.14/wazuh-install.sh
sudo bash ./wazuh-install.sh -a
# This installs the Wazuh indexer, manager, and dashboard on this one host.
# Expect 15-30 minutes depending on your host's disk and CPU allocation.
```

The installer prints an admin username and password at the end of the run copy it, but also retrieve it from the credentials file it generated, since your terminal scrollback is not a reliable place to keep it:

```
sudo tar -O -xvf wazuh-install-files.tar wazuh-install-files/wazuh-passwords.txt
# Look for the block starting "admin" record that password now.
```

Step 2 Reach the Dashboard

- From a browser on your host machine (not inside a VM), navigate to <https://<MERIDIAN-SOC01-IP>/> accept the self-signed certificate warning, this is expected for a lab deployment.
- Log in as admin with the password from Step 1.
- You should land on the Wazuh dashboard's overview, currently showing zero or near-zero agents and events that is correct; you have not onboarded anything yet.

CAPTURE THIS EVIDENCE #1

In your report: a screenshot of the Wazuh dashboard's main overview screen, logged in as admin, showing the version banner and an empty or near-empty agent count. This is your baseline the "before" state the rest of this lab builds on.

Critical thinking #1 This single-node deployment puts the manager, indexer, and dashboard on one host. Referring back to this module's SIEM architecture slide (collection -> parse/normalize -> index/store -> correlation -> case management), name one operational risk of collapsing all of those stages onto a single point of failure, and one reason a real SOC might accept that risk anyway for a smaller deployment.

(3-4 sentences)

3. Part B Onboard Agents

A Wazuh agent is a lightweight collector installed on each monitored host; it is the "collection" stage of the SIEM architecture from lecture, made concrete. You will onboard MERIDIAN-DC01 (Windows) and MERIDIAN-RED01 (Linux) as agents reporting to MERIDIAN-SOC01.

Step 1 Onboard MERIDIAN-DC01 (Windows)

5. On MERIDIAN-DC01, download the current Wazuh agent MSI from <https://packages.wazuh.com/4.x/windows/> (the filename will be wazuh-agent-<version>-1.msi use the current 4.14.x release; the exact patch version changes over time).
6. Install it from an elevated PowerShell window, pointing it at MERIDIAN-SOC01's IP.

```
# In an elevated PowerShell window on MERIDIAN-DC01 (adjust the filename/version and IP):
msiexec.exe /i wazuh-agent-4.14.1-1.msi /q `
  WAZUH_MANAGER="192.168.56.30" `
  WAZUH_AGENT_NAME="MERIDIAN-DC01" `
  WAZUH_REGISTRATION_SERVER="192.168.56.30"

NET START WazuhSvc
```

Step 2 Onboard MERIDIAN-RED01 (Linux)

```
# On MERIDIAN-RED01:
curl -s https://packages.wazuh.com/key/GPG-KEY-WAZUH | sudo gpg no-default-keyring \
  keyring gnupg-ring:/usr/share/keyrings/wazuh.gpg import && \
  sudo chmod 644 /usr/share/keyrings/wazuh.gpg
echo "deb [signed-by=/usr/share/keyrings/wazuh.gpg] https://packages.wazuh.com/4.x/apt/
stable main" | \
  sudo tee /etc/apt/sources.list.d/wazuh.list
sudo apt-get update

WAZUH_MANAGER='192.168.56.30' WAZUH_AGENT_NAME='MERIDIAN-RED01' sudo -E apt-get install -y
wazuh-agent

sudo systemctl daemon-reload
sudo systemctl enable wazuh-agent
sudo systemctl start wazuh-agent
sudo systemctl status wazuh-agent no-pager
```

Step 3 Confirm Both Agents Are Active

- In the Wazuh dashboard, go to Agents (or Endpoint summary, depending on your 4.14 UI layout) both MERIDIAN-DC01 and MERIDIAN-RED01 should show status Active within a minute or two of starting the agent.
- If an agent shows Never Connected or Disconnected, check the manager's IP/port reachability from that host on TCP 1514/1515, and re-check the WAZUH_MANAGER value the agent was installed with.

CAPTURE THIS EVIDENCE #2 AND #3

In your report: a screenshot of the Wazuh dashboard's Agents view showing both MERIDIAN-DC01 and MERIDIAN-RED01 with status Active, including their last-check-in timestamps.

4. Part C Generate Real Telemetry

Pick ONE of the two paths below. Path 1 is the primary path if MERIDIAN-DC01 is a working domain controller (from Lab 4.1 or your own promotion in Section 1.2). Path 2 is fully self-contained and requires nothing but the two agents you just onboarded use it if Path 1's Kerberos setup is not available to you.

Path 1 Re-Trigger the Kerberoasting Burst (Primary)

Confirm Kerberos service-ticket auditing is enabled on MERIDIAN-DC01 Lab 4.1 should have turned this on to produce Event ID 4769 in the first place, but verify it before assuming it.

```
# On MERIDIAN-DC01, elevated PowerShell:
auditpol /get /subcategory:"Kerberos Service Ticket Operations"
# If Success/Failure show "No Auditing", enable it:
auditpol /set /subcategory:"Kerberos Service Ticket Operations" /success:enable
/failure:enable
```

From MERIDIAN-RED01, re-run the SPN enumeration and TGS request burst against the same weak service account Lab 4.1 targeted (adjust the account and domain to match your own Lab 4.1 build):

```
# On MERIDIAN-RED01:
impacket-GetUserSPNs -dc-ip 192.168.56.10 \
'meridian.local/svc_reporting:ChangeMe!2026' -request
```

This requests a TGS for every discoverable SPN a legitimate low-privilege domain account with no special rights is enough to enumerate SPNs and request tickets for them. That is the entire point of Kerberoasting: no elevated access is required to generate this telemetry, only a valid domain logon.

Path 2 Self-Contained SSH Brute Force (Fallback)

Generate a real, detectable burst of failed SSH authentications against MERIDIAN-RED01 from a second terminal on the same host no domain controller required.

```
# On MERIDIAN-RED01, install sshpass if it is not already present:
sudo apt-get install -y sshpass

# Generate 10 failed logins against localhost's own sshd, one every ~3 seconds:
for i in $(seq 1 10); do
  sshpass -p "wrongpass$i" ssh -o StrictHostKeyChecking=no -o ConnectTimeout=3 \
```

```
baduser@127.0.0.1 'exit' 2>/dev/null
sleep 3
done

# Confirm the failures actually landed in the log Wazuh's agent already collects:
sudo tail -n 20 /var/log/auth.log
```

Verify the Telemetry Arrived

- In the Wazuh dashboard, open Discover (or the equivalent raw-event search view) and filter for your source host MERIDIAN-DC01 for Path 1 (look for agent.name and win.system.eventID: 4769), or MERIDIAN-RED01 for Path 2 (look for rule.groups: sshd).
- You should see individual events landing, even though no custom rule has fired an alert yet this is raw collected telemetry, not yet a detection.

CAPTURE THIS EVIDENCE #4

In your report: a screenshot of the raw event(s) in the dashboard's Discover view for Path 1, at least one full 4769 event showing TicketEncryptionType; for Path 2, at least one sshd authentication-failure event showing the source IP and username attempted.

5. Part D Engineer and Test a Detection Rule

This is the deliverable named in the syllabus: write a real Wazuh detection rule, deploy it, and prove empirically that it fires on the behavior from Part C not just that the XML parses. Write the rule that matches the path you generated telemetry for in Part C.

Rule Set 1 Kerberoasting Detection (for Path 1)

This is a two-rule chain: rule 100010 flags any single RC4-encrypted TGS request; rule 100011 uses Wazuh's built-in frequency/timeframe mechanism to escalate when the same source does it 8+ times in 5 minutes the actual kerberoasting indicator, since one RC4 ticket request alone is not necessarily malicious in a legacy-tolerant environment, but a burst from one source is.

```
sudo nano /var/ossec/etc/rules/local_rules.xml
```

On MERIDIAN-SOC01. Add the block below inside a fresh <group> element.

```
<group name="meridian,kerberoasting,attack,">
  <rule id="100010" level="10">
    <if_group>windows_security</if_group>
    <field name="win.system.eventID">^4769$</field>
    <field name="win.eventdata.ticketEncryptionType">^0x17$</field>
    <description>Kerberos TGS request using RC4 encryption</description>
    <mitre>
```

```

        <id>T1558.003</id>
      </mitre>
    <group>kerberoasting,attack_credential_access,</group>
  </rule>

  <rule id="100011" level="12" frequency="8" timeframe="300">
    <if_matched_sid>100010</if_matched_sid>
    <same_source_ip />
    <description>Possible Kerberoasting: 8+ RC4 TGS requests from one source in 5
minutes</description>
    <mitre>
      <id>T1558.003</id>
    </mitre>
  </rule>
</group>

```

local_rules.xml rule IDs 100010-100011 (custom rules must use IDs 100000-120000).

Rule Set 2 SSH Brute-Force Detection (for Path 2)

Wazuh already ships a default multi-failure SSH rule (5720, 8 failures in a window). Write your own, distinct rule here rather than relying on the default a lower, 5-failure threshold in a tighter 2-minute window, so you have engineered and can defend your own choice of threshold.

```

<group name="meridian,ssh_bruteforce,attack,">
  <rule id="100020" level="10" frequency="5" timeframe="120">
    <if_matched_sid>5716</if_matched_sid>
    <same_source_ip />
    <description>Meridian: 5+ SSH authentication failures from one source in 2
minutes</description>
    <mitre>
      <id>T1110.001</id>
    </mitre>
  <group>ssh_bruteforce,attack_credential_access,</group>
</rule>
</group>

```

local_rules.xml rule ID 100020, chained off the built-in sshd auth-failure rule 5716.

Deploy and Test

```

sudo /var/ossec/bin/wazuh-logtest -t
# Confirm no syntax errors are reported before restarting.

sudo systemctl restart wazuh-manager

```

Now re-trigger the exact behavior from Part C re-run the `impacket-GetUserSPNs` command (Path 1) or the `sshpas` loop (Path 2) again and check the dashboard.

CAPTURE THIS EVIDENCE #5

In your report: a screenshot of your fired alert in the Wazuh dashboard, showing the rule ID and description you wrote, the rule level, and the <mitre><id> tag rendered in the alert's detail view.

CAPTURE THIS EVIDENCE #6

In your report: a screenshot of the underlying individual events that fed your frequency rule enough of them visible in Discover to show the same source IP repeating within your rule's timeframe window.

Critical thinking #2 Rule 100011 (or 100020) uses if_matched_sid plus frequency and timeframe rather than a single field match. Explain specifically what would happen, detection-wise, if you had written only rule 100010 (or only the single-event sshd rule) and deployed it alone without the frequency-based escalation rule what would still fire, and what genuinely kerberoasting-indicative signal would you lose?

(4-5 sentences this is the core distinction between an event-match rule and a behavior-match rule)

6. Part E Alert Triage Exercise

Meridian's Tier 1 analyst queue received the eight alerts below overnight, from across Meridian's full environment (not just your lab hosts). For each one, apply the Tier 1 triage model from lecture: is it a true positive or false positive, does it map to an ATT&CK technique, and what is the correct next action escalate to Tier 2, close, or flag for tuning?

#	Time	Source	Rule Fired	Raw Context
1	02:14:07	BILL-WS-14	100011 Kerberoasting burst	9 TGS requests (0x17) for svc_reporting in 4m 40s from one workstation, outside business hours.
2	09:31:52	NUR-CLIN-014	5720 SSH multi-auth-failure	8 failed SSH logins to a Linux print server in 45 seconds, all as user 'root', then a successful login.
3	13:02:18	vpn-gw-01	100020 SSH brute-force (custom)	6 failed SSH attempts across 90 seconds from a remote-access VPN pool IP, different usernames each attempt, no success.
4	13:47:03	MERIDIAN-DC01	100010 RC4 TGS request	Single RC4 TGS request for a legacy print-server SPN, requested by a helpdesk account

#	Time	Source	Rule Fired	Raw Context
				during a scheduled maintenance window.
5	18:20:44	corp-backup-01	100020 SSH brute-force (custom)	7 failed SSH attempts in 100 seconds from an internal backup-server IP, same username each time, then success known nightly backup job re-authenticating after a credential rotation.
6	21:55:31	ADMIN-WKS-03	100011 Kerberoasting burst	11 TGS requests (0x17) for 4 different SPNs in 3 minutes from a domain admin's workstation, immediately after the admin reported their laptop 'acting strange.'
7	23:10:09	guest-ap-12	5720 SSH multi-auth-failure	9 failed SSH logins in 30 seconds from a guest Wi-Fi IP against a host with no SSH service actually listening on that port.
8	23:59:58	svc-batch-07	100010 RC4 TGS request	3 separate single RC4 TGS requests, 40 minutes apart, all for the same SPN, from a known nightly ETL service account.

For each of the 8 alerts, provide, in a table of your own:

- TP or FP your classification, and one sentence of justification grounded in the specific evidence given.
- ATT&CK technique ID, if one applies or "none" if the alert does not map to a technique.
- Disposition escalate to Tier 2, close as benign, or flag the rule itself for tuning.

Critical thinking #3 Using only alerts #1 and #6 (both fired by the same rule, 100011), compute MTTD in minutes for each: treat the moment named in "Raw Context" as the true attack start, and the alert's listed Time as the detection moment. State both numbers and, in 2-3 sentences, explain why the same rule can produce two very different MTTD figures depending on when within the attack's own window it happens to cross the frequency threshold.

(Show your arithmetic, not just the final numbers)

7. Challenge Tune Without Missing (Required)

THE STATED FALSE-POSITIVE SCENARIO

Your Part D SSH brute-force rule (100020: 5+ failures from one source in 120 seconds) currently fires on a legitimate but slow pattern: a remote employee's VPN client silently retrying a stale cached SSH key against a jump host 5-6 times over about 100 seconds during a flaky connection, before falling back to the correct key and succeeding. This happens for real users several times a week and is not an attack.

Tune rule 100020 to eliminate this specific false positive, without raising the frequency threshold so high that it would also fail to catch the original attack scenario from Part C (5+ rapid failures, no eventual legitimate success). You have two realistic levers, and you may use one or combine both defend whichever you choose:

- Add a same-username condition (Wazuh's `<same_field name="..."/>` or a second `<field>` match) so the rule only fires when the failures are against a single account, not merely a single source IP a VPN reconnect retries one known account; an actual brute force is often guessing across multiple usernames.
- Add a check-not-followed-by-success window in your triage logic (a Tier 1 SOP note, since Wazuh's rule engine alone cannot easily express "and no success afterward") that tells an analyst to check the immediately following event for that same source before escalating.

Prove both properties with evidence: (a) re-run a simulated version of the benign VPN-retry pattern (same username, same source, 5-6 failures then one success within 100 seconds) and show your tuned rule does NOT fire; (b) re-run your original Part C attack pattern and show your tuned rule STILL fires.

CAPTURE THIS EVIDENCE #7 AND #8

In your report: your tuned rule's full XML, a screenshot proving the benign VPN-retry simulation does not raise your rule (Discover shows the raw events, no corresponding alert), and a screenshot proving the original attack pattern still does.

Critical thinking #4 (synthesis) This challenge is Section 2's lifecycle step 5 (tune) in miniature. In 4-5 sentences, name the specific false-positive-rate cost of leaving rule 100020 untuned across Meridian's whole environment (tie it to the alert-fatigue argument from lecture whose time does an untuned rule actually spend?), and state one way you would validate your tuned rule stays correct if Meridian's VPN infrastructure changes six months from now.

(4-5 sentences)

8. What to Submit

Submit a single Word or PDF lab report not this document filled in, a document you write containing:

- All 8 pieces of captured evidence (screenshots), each clearly labeled Evidence #1 through #8

- Written answers to all 4 critical-thinking prompts, each labeled to match its number
- Your full local_rules.xml additions from Part D, and your tuned rule from the Challenge
- Your Part E triage table for all 8 alerts, plus the MTTD calculation from Critical Thinking #3
- A short (1 paragraph) note on which fork you took at each choice point (Path 1 vs. Path 2 in Part C; Lab 4.1 reuse vs. fallback setup in Section 1) and why

File name: LastName_Week04_Module02_Lab.docx (or .pdf). Submit by the Week 04 deadline posted in Canvas.

Submission Checklist

- ☐ MERIDIAN-SOC01 deployed, dashboard reachable, Evidence #1 captured
- ☐ Both agents (DC01/fallback Windows host, RED01) showing Active, Evidence #2-3 captured
- ☐ Real telemetry generated and verified in Discover, Evidence #4 captured
- ☐ Custom detection rule written, deployed, and fired successfully, Evidence #5-6 captured
- ☐ Part E triage table completed for all 8 alerts, with MTTD arithmetic shown
- ☐ Challenge: tuned rule proven to suppress the FP AND still catch the original attack, Evidence #7-8 captured
- ☐ All 4 critical-thinking prompts answered

9. Grading Rubric (100 pts)

Criterion	Pts	Distinguished	Proficient	Developing	Insufficient
SIEM deployment & agent onboarding	15	Manager, indexer, dashboard, and both agents verified with clear, correctly-labeled evidence.	Deployment complete; evidence present but missing minor detail (e.g., timestamps).	Deployment complete but one agent's evidence is missing or unclear.	SIEM or agents not demonstrably working.
Telemetry generation	10	Real telemetry generated and clearly verified in raw event search with correct field values shown.	Telemetry generated and verified, minor gaps in evidence.	Telemetry generated but verification evidence is weak or ambiguous.	No credible evidence telemetry reached the SIEM.
Custom detection rule (Part D)	25	Correct, well-formed XML; correct ATT&CK mapping; rule demonstrably fires with clear evidence.	Rule fires correctly; minor XML or mapping imperfection.	Rule deployed but does not clearly fire, or ATT&CK mapping is wrong.	No working custom rule submitted.
Alert triage table (Part E)	15	All 8 correctly classified and justified with specific evidence; MTTD arithmetic correct and shown.	6-7 correctly classified; MTTD correct.	Several misclassifications or MTTD arithmetic error.	Table missing or mostly incorrect.
Challenge: tuning (Section 7)	20	Tuned rule proven with evidence to	Tuning correct in concept; one of	Tuning attempted but does not	Challenge not attempted or

Criterion	Pts	Distinguished	Proficient	Developing	Insufficient
		suppress the exact FP while still catching the attack.	the two evidence proofs is weak.	clearly solve the stated problem.	fundamentally misunderstood.
Critical thinking responses	15	All 4 answers precise, technically grounded, and specific to captured evidence.	Answers correct but somewhat generic.	Some answers vague or partially incorrect.	Most answers missing or incorrect.

10. References

- Wazuh, Inc. Quickstart All-in-one Installation. <https://documentation.wazuh.com/current/quickstart.html>
- Wazuh, Inc. Custom Rules Ruleset XML Syntax. <https://documentation.wazuh.com/current/user-manual/ruleset/rules/custom.html>
- Wazuh, Inc. Rules Syntax Ruleset XML Syntax. <https://documentation.wazuh.com/current/user-manual/ruleset/ruleset-xml-syntax/rules.html>
- Wazuh, Inc. Deploying Wazuh Agents on Windows/Linux Endpoints. <https://documentation.wazuh.com/current/installation-guide/wazuh-agent/index.html>
- SigmaHQ. Sigma Specification and Rule Repository. <https://sigmahq.io/>
- MITRE Corporation. MITRE ATT&CK, T1558.003 Steal or Forge Kerberos Tickets: Kerberoasting. <https://attack.mitre.org/techniques/T1558/003/>
- MITRE Corporation. MITRE ATT&CK, T1110 Brute Force, and T1110.001 Password Guessing. <https://attack.mitre.org/techniques/T1110/>
- National Institute of Standards and Technology. SP 800-92, Guide to Computer Security Log Management.
- Open Cybersecurity Schema Framework Project. OCSF Documentation. <https://ocsf.io/>
- Impacket Project (Fortra). GetUserSPNs.py. <https://github.com/fortra/impacket>

Lab 4.2, Week 04 Detection Engineering, SIEM & SOC Operations IA 642, Defensive Security. Continues the Meridian Health Network scenario from this module's lecture and Module 01's Lab 4.1.